
RQ-01 Data Storage Foundation

Mục tiêu

Xây dựng nền tảng lưu trữ dữ liệu dạng Knowledge Graph tối giản phục vụ AI.

Nguyên tắc

- Chỉ có 2 loại dữ liệu:
 - Node
 - Relationship
 - Mọi nội dung đều là text.
 - Không ép schema theo domain.
 - Tối ưu cho AI đọc/ghi.
 - Tối ưu traceability.
 - Tối ưu mở rộng sau này.
 - Chưa cần search thông minh.
 - Chưa cần AI retrieval.
 - Chưa cần visualization graph.
-

Checklist Implementation

1. Node Model

Mục đích

Lưu trữ mọi tri thức của hệ thống.

Cần code

```
1 Node
2   ├── id
3   ├── title
4   ├── content
5   ├── version
6   ├── createdAt
7   └── updatedAt
```

Yêu cầu đạt được

- ✓ Tạo Node

- ✓ Đọc Node
 - ✓ Cập nhật Node
 - ✓ Xóa Node
 - ✓ Mỗi Node có ID duy nhất
 - ✓ Không phụ thuộc loại dữ liệu
-

2. Relationship Model

Mục đích

Lưu kết nối giữa các Node.

Cần code

```
1 Relationship
2   | id
3   | sourceNodeId
4   | targetNodeId
5   | metadata
6   | version
7   | createdAt
8   | updatedAt
```

Yêu cầu đạt được

- ✓ Một Relationship luôn nối 2 Node
- ✓ Metadata là text tự do
- ✓ Hỗ trợ nhiều metadata

Ví dụ:

```
1 đồng đội
2 thầy trò
3 vừa yêu vừa hận
```

- ✓ CRUD đầy đủ
-

3. Node Repository

Mục đích

Quản lý lưu trữ Node.

Cần code

```
1  CreateNode()  
2  
3  GetNode()  
4  
5  UpdateNode()  
6  
7  DeleteNode()  
8  
9  ListNodes()
```

Yêu cầu đạt được

- ✓ Không cần filter phức tạp
 - ✓ Có thể lấy toàn bộ Node
 - ✓ Có thể lấy theo ID
-

4. Relationship Repository

Mục đích

Quản lý lưu trữ Relationship.

Cần code

```
1  CreateRelationship()  
2  
3  GetRelationship()  
4  
5  UpdateRelationship()  
6  
7  DeleteRelationship()  
8  
9  ListRelationships()
```

Yêu cầu đạt được

- ✓ CRUD đầy đủ
 - ✓ Lấy được relationship theo ID
 - ✓ Lấy được toàn bộ relationship
-

5. Node Index

Mục đích

Tìm Node nhanh theo tên.

Cần code

```
1 NodeTitleIndex
```

Ví dụ

```
1 "Luffy" → N001
2
3 "Zoro" → N002
```

API

```
1 FindNodeByTitle()
```

Yêu cầu đạt được

- ✓ Không phải scan toàn bộ Node
 - ✓ Một title trả về Node ID
-

6. Relationship Index

Mục đích

Tăng tốc traceability.

Cần code

```
1 NodeId
2 ↓
3 RelationshipIds
```

Ví dụ

```
1 N001
2
3 → R001
4 → R005
5 → R100
```

API

```
1 GetRelationshipsOfNode()
```

Yêu cầu đạt được

- ✓ Lấy toàn bộ relation của Node
 - ✓ Không cần scan full database
-

7. Graph Traversal Service

Mục đích

Cho phép đi từ Node sang các Node liên quan.

Cần code

```
1 GetConnectedNodes(nodeId)
```

Flow:

```
1 Node
2 ↓
3 Relationship
4 ↓
5 Connected Node
```

Yêu cầu đạt được

- ✓ Lấy node liên kết trực tiếp
- ✓ Hỗ trợ inbound

```
1 A -> B
```

- ✓ Hỗ trợ outbound

```
1 B <- A
```

8. Persistence Layer

Mục đích

Đảm bảo dữ liệu tồn tại sau khi restart hệ thống.

Cần code

NGUYEN TAC

- Lưu trữ theo cấu trúc Project.
- Mỗi Project là một không gian world building độc lập.
- Dữ liệu giữa các Project không ảnh hưởng nhau.
- Phase 0 sử dụng JSON File Storage.
- Dữ liệu gồm Node, Relationship và các Index liên quan.
- Mỗi nội dung được lưu dạng text.

PROJECT STRUCTURE

Project

- | - project.json
- | - nodes/
- | - relationships/
- | - indexes/
 - | | - node-title-index.json
 - | | - relationship-index.json

Yêu cầu đạt được

- ✓ Dữ liệu không mất
 - ✓ Có thể load lại graph
 - ✓ Có thể rebuild index
-

9. Versioning Foundation

Mục đích

Chuẩn bị cho history sau này.

Cần code

Trong Node:

```
1 version
```

Trong Relationship:

```
1 version
```

Yêu cầu đạt được

- ✓ Mỗi lần update tăng version
 - ✓ Chưa cần lưu lịch sử
 - ✓ Chỉ cần track version hiện tại
-

10. Integrity Validation

Mục đích

Ngăn graph bị hỏng.

Cần code

Validation:

```
1 CreateRelationship()
```

phải kiểm tra:

```
1 source tồn tại
2
3 target tồn tại
```

Yêu cầu đạt được

- ✓ Không có relation trỏ tới node không tồn tại
 - ✓ Không có ID trùng
-

11. Import / Export

Mục đích

Backup dữ liệu.

Cần code

```
1  ExportNodes()  
2  
3  ExportRelationships()  
4  
5  ImportNodes()  
6  
7  ImportRelationships()
```

Yêu cầu đạt được

- ✓ Xuất toàn bộ graph
 - ✓ Khôi phục từ backup
-

12. Rebuild Index Service

Mục đích

Nếu index bị mất vẫn khôi phục được.

Cần code

```
1  RebuildNodeIndex()  
2  
3  RebuildRelationshipIndex()
```

Yêu cầu đạt được

- ✓ Quét toàn bộ storage
 - ✓ Tạo lại index
 - ✓ Không mất dữ liệu
-

Acceptance Checklist (Definition of Done)

Khi hoàn thành RQ-01, tối thiểu phải check được:

Data Model

- Node model
- Relationship model

Data Storage

- Node persistence
- Relationship persistence

CRUD

- Create node
- Read node
- Update node
- Delete node
- Create relationship
- Read relationship
- Update relationship
- Delete relationship

Index

- Node index
- Relationship index
- Rebuild node index
- Rebuild relationship index

Graph

- Get relationships of node
- Get connected nodes
- Support inbound traversal
- Support outbound traversal

Integrity

- Unique ID validation
- Relationship reference validation

Persistence

- Restart không mất dữ liệu

- Export dữ liệu
- Import dữ liệu

Future Ready

- Version field trên Node
 - Version field trên Relationship
-

Nếu là Tech Lead review requirement này thì tôi sẽ coi **RQ-01 hoàn thành** khi hệ thống đã có thể tạo một graph tri thức cơ bản, lưu được xuống storage, load lại được, truy vết được từ một node đến tất cả node liên quan, và có đủ index để làm nền cho search/retrieval ở các phase tiếp theo. Đây là toàn bộ phần "storage foundation", chưa dính gì tới AI inference hay search logic nâng cao.